ECS Pre-Trip Travel Request — Technical Reference

Audience: RFP respondents / build team **Basis:** working validation prototype **Generated:** 2026-10-22

REQUEST FOR PROPOSAL — TECHNICAL APPENDIX

Document	Technical Reference for RFP respondents / implementation build team
Module	ECS Pre-Trip Travel Request (extension of the NTU Employee Claim System)
Version	1.0 — draft for review
Basis	Working validation prototype — Next.js 16 · React 19 · TypeScript · Prisma 6 · SQLite
Subject	Pre-trip travel request, approval, TMC hand-off and expense-claim seeding built on the existing ECS/ECS-Travel platform

> This document describes the behaviour of a requirements-validation prototype. It is complete enough to be read as an **executable specification** — every rule, algorithm and field below is implemented in the prototype and demonstrable. Statements marked **Derived requirement (TR-PT-nn)** are proposals derived from that behaviour and remain subject to confirmation by the University. The prototype is a reference for *what* must be built, not a mandate for *how*. Where this document and the demonstration differ, this document records the intended behaviour; Chapter 14 records the places where the prototype deliberately simplifies.

How to read this document

READER	WHAT THEY NEED	START WITH
Solution architect	Scope, boundary, layering, integration, production gaps	Ch 1–3, 9, 14
Data / integration engineer	Entity model, field dictionary, versioning, interface payloads	Ch 4, 5, 9, 12
Application developer	Runtime algorithms, service & action contracts, error model, state machines, screens	Ch 6, 7, 10, 11
Security / controls reviewer	Separation of duties, access derivation, scope, audit, enforcement gaps	Ch 8, 14
Test lead / evaluator	Acceptance criteria, test inventory, open decisions, demo script	Ch 13, 14, App. D
Procurement panel	Scope, derived requirements, gap register, NFRs	Ch 1, 2, 14

Conventions. Keywords **MUST / SHOULD / MAY** are used in the RFC sense. A *Derived requirement* (TR-PT-nn) states a requirement implied by prototype behaviour. A *Design decision* (DD-nn) is a

binding decision documented in the companion design Build Spec (App. D). A *Decision point* (O-nn) is an open policy item for the University to confirm. Code identifiers are in `monospace`. Money is SGD; the prototype uses floating SGD but production **MUST** use exact decimal (TR-PT-04). Dates are stored as `DateTime`; comparisons use the official travel dates unless stated.

Chapter 1 — Introduction

1.1 Purpose

This is the technical appendix to the RFP for the **ECS Pre-Trip Travel Request** module. It describes, at developer level: the data the module owns and how it links; the configuration that expresses NTU travel policy; the algorithms that derive entitlement, cost, policy outcomes and the approval route; the controls that govern submission, approval and change; and the boundaries with neighbouring systems (ECS core, HR/Workday, DOA, SAP Finance, the TMC/OBT, and the downstream TE claim).

1.2 Scope & system boundary

In scope (built by this module): the pre-trip lifecycle from *draft request* to *approved Travel Authorisation*, the governed *TMC hand-off*, *booking reconciliation*, and the *seeding of the downstream TE claim*. The module owns the transactional store in §4.2 and the runtime Module Settings (§5.1), the Integration-Contract overrides (§9) and the Provost list. **Out of scope (reused read-only from ECS):** HR identity & appointments, reporting lines, DOA lines & amount bands, charging accounts (CC/WBS), policy caps/ODA/class register, FX rates, the location/airport master, TMC providers, GST/GL codes. The module **reads** these through the single service boundary in §2 (Ch 10) — it never maintains them. **Downstream:** the TE claim is created *from* the Travel Request (Ch 6.13, Ch 10) but the claim's own post-trip lifecycle beyond create/pre-populate/auto-grant is the ECS TE module's.

1.3 What is real and what is simulated

AREA	REAL IN THE PROTOTYPE	SIMULATED / TO BE WIRED IN PRODUCTION
Lifecycle, routing, policy, cost, settlement, duplicate, class entitlement	Fully implemented, server-enforced	—
Reference masters (employees, DOA lines, caps, ODA, bands, FX, locations)	Seed data read through the §2 services	Replace with live ECS/HR/DOA/Finance reads
TMC hand-off & inbound	Canonical payload built; inbound **simulated** by an adapter	Replace the adapter with the real TMC/OBT interface
SAP airfare posting, visa-letter batch, notifications	Modelled as actions/audit	Wire to SAP, immigration, notification channels
File storage (attachments)	Bytes stored in-row	Replace with a document store
Authentication	Persona cookie switcher	Replace with ECS SSO/identity (§8.2)

1.4 Provenance

The requirements are derived from the ECS Pre-Trip Build Pack and the CFO review decisions (DD-01…DD-29, App. D), validated by the running prototype. Companion artifacts: the **design Build Spec** (decisions + screen narratives) and the **Feature Walkthrough** (live screens).

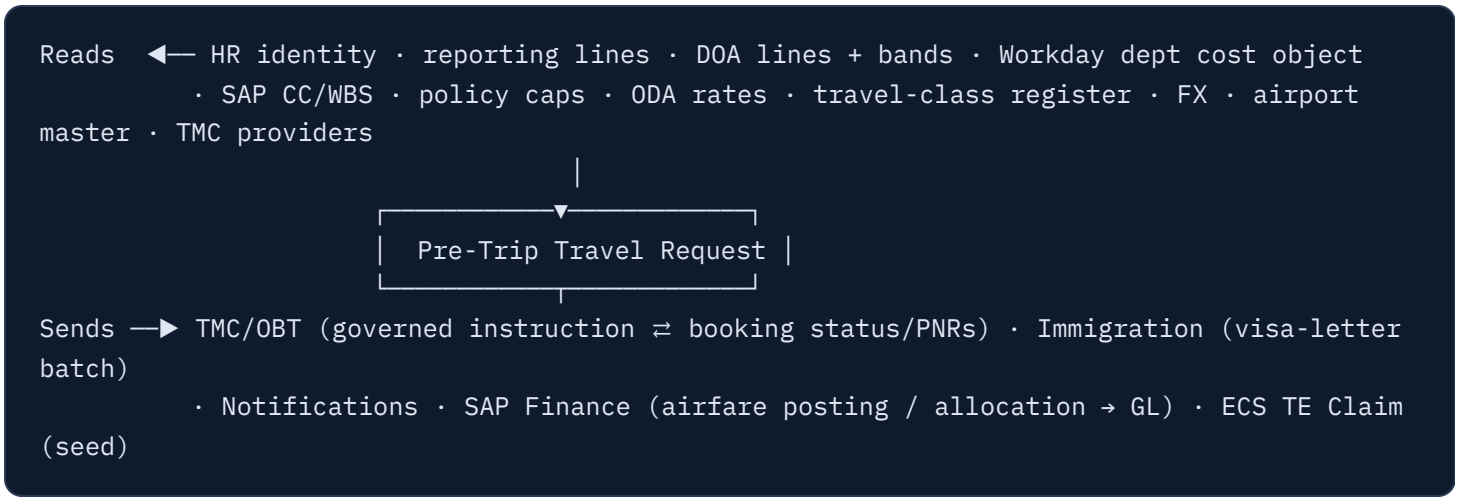
TR-PT-01 The solution MUST implement the full pre-trip lifecycle (draft → approval → Travel Authorisation → TMC hand-off → booking reconciliation → claim seeding) with policy and routing enforced **server-side**, never trusting the client. **TR-PT-02** The solution MUST read all reference masters (identity, DOA, charging, policy rates, FX, locations, bands) through a single integration boundary and MUST NOT maintain copies of them.

Chapter 2 — Solution overview

2.1 What the module does

It closes the gap between an employee's **intent to travel** and a **booked, policy-compliant, financially authorised trip**: capture the trip; derive entitlement (travel class) and cost (airfare, cap-checked accommodation, tiered ODA); evaluate policy (caps, class, duplicates, risk); route for approval (configurable RO / Additional / Funding / Cross-BA / Exception / Provost / **DOA**); issue a dated, amount-bound **Travel Authorisation (TA)**; transmit a governed booking instruction to the TMC; reconcile the booking (PNRs, fares, deviation); and seed the post-trip **TE claim** so it reconciles to what was approved.

2.2 System context



2.3 Actors / personas

`ROLE` = `TRAVELLER, TRAVEL_REQUESTOR, RO, DOA, RESEARCH_DOA, EXCEPTION_APPROVER, TRAVEL_ADMIN, SYSTEM_ADMIN, FINANCE_VIEWER` (plus the guest **Provost** authority, DD-25). The prototype's persona switcher (`PERSONA_ROLES` , grouped into REQUEST / CLAIM / ADMIN stages) selects who you act as; in production this is ECS SSO (§8.2). **Creation remit** and **approver entitlement** are role-derived and enforced server-side (§8).

2.4 Core concepts

CONCEPT	MEANING	REFERENCE
Travel Request (TR)	The governed transactional record; `TR-YYYY-NNNNNN`.	§4, Ch 6
Travel Authorisation (TA)	Dated, amount-bound authority issued on final approval; `TA-YYYY-NNNNNN`.	§6.9
Estimate line	Airfare / Accommodation / ODA / Conference / Other; ODA + hotel are **auto-derived per city**.	§6.1
Approval route	Ordered steps computed from context + Module Settings; always ends at a DOA.	§6.6
Policy check	PASS / WARNING / EXCEPTION / HARD_STOP outcome per rule.	§6.4
Settlement channel	Who fronts the money (CTA/central vs employee) → reimbursable vs reference-only (DD-19).	§6.10
Booking / PNR	One TR → **N PNRs** (multi-city split); retrieval is **TA-keyed** (DD-17).	§6.12, §9.4

2.5 Capability map

Request capture (individual / group / guest) · itinerary & trip types (round / one-way / multi-city, personal segments) · entitlement derivation · cost estimation (auto per-city hotel + tiered ODA, airfare, FX, sponsorship) · policy engine · configurable routing · charging & allocation · authorisation & booking lifecycle · governed TMC hand-off + group fan-out + multi-PNR · claim seeding + DOA auto-grant · amendments, versioning & audit · runtime configuration.

2.6 Guardrails & design principles

- **Server-authoritative.** Every mutation re-derives and re-checks; the UI is a convenience, not a gate (§8).
- **Separation of duties.** No one approves a request they travel on or raised; approver-only personas cannot originate a request (§8.6, DD-28).
- **Configuration over code.** Routing, thresholds, caps basis, tolerances and the integration contract are runtime settings (§5).
- **One live authorisation per trip.** A true duplicate is blocked; surfaced upfront (DD-14, §6.11).
- **Policy before spend.** Caps, class and exceptions are evaluated and routed before booking, not discovered at claim.
- **One TA is the single source of truth** the TMC and the claim both reconcile against.

TR-PT-03 Approval authority (RO/DOA/Exception) MUST NOT be exercisable by a Travel or System Administrator by virtue of their admin role; an administrator may approve only when they independently hold the role **and** are the assigned approver.

Chapter 3 — Architecture reference

3.1 Technology stack (prototype)

Next.js 16 (App Router, Turbopack), React 19 **Server Components + Server Actions**, TypeScript, Prisma 6 ORM, SQLite. Tailwind v4 styling. No REST API layer — **all mutations are server actions** (`"use server"`); all reads are server-component data loaders. Production mapping:

Screens/Blocks ← pages; Server Actions ← server actions; Entities ← Prisma models; Integration (REST / Service Actions) ← the §2 ECS services.

3.2 Layered architecture

LAYER	RESPONSIBILITY
Presentation	Wizard (Traveller→Trip→Estimates→Charging→Policy→Submit), approver/DOA queue, TE claim, admin/config, finance & reporting.
Domain logic	Route engine, class entitlement, cost/ODA/accommodation, policy evaluation, validation, charging roll-up, settlement, duplicate detection, claim workflow & auto-grant.
Integration seam	`EcsIdentity` · `EcsRoles` · `EcsDelegation` · `EcsReference` · `EcsFx` · `EcsPolicy` · `EcsTravelClassRegister` · `EcsCharging` · `EcsWorkflow` + TMC adapter + notification/visa hooks.
Transactional store	§4.2 entities.
Reference masters	Read-only ECS masters — never written.

3.3 Module map (source)

`src/modules/pretrip/*` — `actions.ts` (server actions), `route.ts` (routing), `policy.ts` (policy engine), `pricing.ts` (cost), `settlement.ts` (DD-19 + guest settlement), `duplicates.ts` (DD-14), `tmcPayload.ts` / `tmcRetrieval.ts` / `tmcRouting.ts` / `handoffRouting.ts` (TMC), `emergency.ts`, `risk.ts`, `provost.ts`, `group.ts`, `chargingRollup.ts`, `guards.ts` (access), `queries.ts` (loaders), `numbering.ts`, `settings.ts`, `demo.ts` (seed). `src/modules/te/*` — claim `actions.ts`, `prepopulate.ts`, `workflow.ts`. `src/shared/ecs/services.ts` — the integration seam. `src/config/*` + `src/data/*` — settings catalogue + reference masters. `src/integrations/tmc/*` — adapter + canonical contract. Full source map in App. C.

3.4 Request lifecycle & trust boundaries

Every server action: (1) resolves the acting persona (`currentPersonaId`); (2) checks the lifecycle source state (`actionAllowed`) and the acting remit (`canEditRequest` / `canActOnStep`); (3) mutates in a transaction; (4) writes an `AuditEvent`; (5) `revalidatePath` (soft refresh, preserves client state) or `redirect` (for a validation failure, with `?error=`). The trust boundary is the server action — the client never carries authority.

3.5 Error model

- **Validation failures** redirect back to the step with `?error=<message>` (user-facing, specific).
- **Hard stops** are `PolicyCheck` rows that disable Submit (not thrown).
- **Guard failures** (wrong state/remit) no-op and audit; actions return without effect.
- **Seed integrity** assertions throw loudly so bad seed data is caught at build.

3.6 Build, run & seed

`npm run dev`; `npm run seed` seeds the single `ModuleSettings` row; demo scenarios (28, §12.2) seed lazily when the request table is empty. **Migrations are additive** — `prisma db push` + a marked migration via `migrate diff … --script` + `migrate resolve --applied`; never `migrate dev`/reset against seeded data. SQLite has no native enum — status/type columns are `String`; production MAY use native enums or check constraints.

3.7 Architectural requirements for production

TR-PT-05 All approval, policy and routing logic MUST execute server-side and be re-evaluated on submit and on every approval, independent of UI state. **TR-PT-06** Money MUST be exact decimal; FX MUST be effective-dated and re-locked at submission. **TR-PT-07** Every material action MUST write an append-only audit event; approved/submitted states MUST snapshot an immutable version. **TR-PT-08** The TMC hand-off and inbound ingestion MUST be idempotent (a booking already sent/in-flight/booked is never re-sent or duplicated).

Chapter 4 — Data architecture

4.1 Modelling conventions

CONVENTION	RULE
Row identity	Every row has a `cuid()` surrogate `id`; the TR also carries a natural number `requestNumber` (`TR-YYYY-NNNNNN`) and, once approved, `authorisationNo` (`TA-YYYY-NNNNNN`), both unique.
References	Transactional children hold a formal FK to their parent (`requestId`, `expenseId`, `bookingId`, `claimId`) with **cascade delete**. References to **reference masters** (employee ids, charging codes, city/country/airport codes, class ids, expense-type ids) are **soft** — stored as the natural code and resolved through the §2 services.
Enumerations	SQLite has no enum: status/type fields are `String` constrained in the application to the values in §4.4 / App. B.
Structured values	JSON held as text in a `String` column: `shareMap`, `allocJson`, `chargingJson`, integration `payload`, version `snapshot`. Production SHOULD use JSON/JSONB or normalised child tables.
Effective dating	Reference masters (caps, ODA, FX, class register) are effective-dated and read as-at the submission/entry date. Transactional records are versioned by `TravelRequestVersion` snapshots on submission/approval; the TR itself carries `currentVersion`.
Money & dates	Amounts are SGD (prototype `Float`; production MUST be exact decimal). Dates are `DateTime`; the **official** `startDate`/`endDate` are the governing dates for duplicates, ODA days and validation.
Append-only	`AuditEvent` and `IntegrationMessage` are write-and-read only (no update/delete route).
Correlation	Audit and integration rows carry `requestId`; a booking's PNRs are correlated to the TA (`authorisationNo`).

TR-PT-09 Submitted and approved states MUST be snapshotted as immutable versions; a superseded snapshot MUST NOT be overwritten. **TR-PT-10** Audit and integration-message records MUST be append-only with no update/delete interface.

4.2 How the data links together

```
TravelRequest 1–* TravelRequestTraveller        (group members; per-traveller class & sub-itinerary)
              1–* ItineraryLeg                   (shared or per-traveller legs)
              1–* EstimatedExpense 1–1 Accommodation | ODA   (auto-derived per city)
              1–* ChargingAllocation             (CC/WBS split = 100%)
              1–* PolicyCheck                    (engine output)
              1–* ApprovalStep                   (computed route)
              1–* TravelBooking 1–* BookingSegment (one TR → N PNRs)
              1–* BookingDeviation
              1–* IntegrationMessage · AuditEvent · Attachment · TravelRequestVersion
              1–* TravelExpenseLink 1–1 TravelExpenseClaim 1–* TEExpenseLine · ClaimApprovalStep
```

Reference-master joins (soft, by code): `employee`, `department`, `chargingCode`, `city/country/airport`, `travelClass`, `expenseType`, `workflowBand`, `odaRate`, `hotelCap`, `fx`.

4.3 Entity catalogue (field-level data dictionary)

Format: **Field · Type · Constraint · Description**. `?` = nullable. All children cascade-delete from `TravelRequest` unless noted.

4.3.1 `TravelRequest` — header & lifecycle

FIELD	TYPE	CONSTRAINT	DESCRIPTION
id	String	PK, cuid	Surrogate key.
requestNumber	String	unique	`TR-YYYY-NNNNNN`.
authorisationNo	String?	unique	`TA-YYYY-NNNNNN`; set on final approval.
status	String	default `Draft`	Request state (§4.4).
bookingStatus	String	default `Not Sent`	Booking state (§4.4).
isGroup	Boolean	default false	Group request flag.
requestorId	String		Creator (may be non-traveller).
travellerId	String		Primary traveller; `GUEST` for a non-employee.
travellerType	String	default EMPLOYEE	`EMPLOYEE | GUEST`.
guestName / guestEmail / guestOrg	String?		Guest identity (no HR record; PDPA — §9).
additionalApproverId	String?		Traveller-selected extra approver.
selectedDoaId	String?		Traveller-selected DOA (not pre-selected).
provostId	String?		Selected Provost for a guest class upgrade.
isEmergency / isRetrospective	Boolean	default false	Emergency lane flags (DD-23).
emergencyJustification / emergencyAuthoriserId	String?		Emergency reason + authorising officer.
emergencyRatified / emergencyOutsideWindow	Boolean	default false	Retrospective ratification / out-of-window flag.
entityId / departmentId	String?		Owning entity / department.
isResearch	Boolean	default false	Research-charged (→ Research DOA).
purposeId / description	String?		Trip purpose + justification.
tripType	String	default ROUND_TRIP	`ROUND_TRIP | ONE_WAY | MULTI_CITY`.
destCountry / destCity	String?		Derived main destination (longest-stay leg).
startDate / endDate	DateTime?		**Official travel dates** (governing).

FIELD	TYPE	CONSTRAINT	DESCRIPTION
personalStart / personalEnd	DateTime?		Personal extension window.
personalAck	Boolean	default false	Personal-portion-at-own-cost ack.
highRiskAck / highRiskApproverAck	Boolean	default false	§4.8 acknowledgements.
visaLetterRequired	Boolean	default false	Visa-letter needed (§4.13).
visaLetterNotifiedAt	DateTime?		Set by the daily visa batch.
tmcProviderId	String?		Preferred TMC provider (null = auto).
eventStartDate / eventEndDate	DateTime?		Event/conference dates (may differ from travel).
invitationRef	String?		Invitation/acceptance reference.
travelClassId / entitledClassId / classBasis / classJustification	String?		Chosen + derived class (individual) + basis + upgrade justification.
bookingArrangement	String?	default AUTO	`AUTO | TMC-<id> | SELF_BOOKED | HOST_ARRANGED | NO_BOOKING`.
bookingMethod	String?		TMC channel `TMC_ONLINE | AGENT_ASSISTED | OFFLINE` (when TMC-arranged).
chargingMode	String	default REQUEST	`REQUEST | LINE`.
hotelSettlement	String	default INDIVIDUAL	**DD-19** `INDIVIDUAL | CTA` — hotel payment channel (§6.10).
approvalAmountSgd	Float?		Amount-basis result for DOA tiering.
bookingDeadline / authorisationExpiry	DateTime?		§13.3 booking window / TA validity.
currentVersion	Int	default 0	Version counter.
createdAt / updatedAt	DateTime	now / @updatedAt	

4.3.2 `TravelRequestTraveller` — group member

`id` PK · `requestId` FK · `employeeId` (or synthetic `G-…` for a guest member) · `travellerType` (EMPLOYEE|GUEST) · `guestName/Email/Org?` · `confirmed` + `confirmedAt?` (inclusion confirmation) · `sharePct` (default 100) · `isRequestor` · `entitledClassId?/chosenClassId?/classBasis?/classJustification?` (per-traveller class — DD-05) · `ownStartDate?/ownEndDate?/legOverrideNote?` (per-traveller sub-itinerary — §13.20).

4.3.3 `ItineraryLeg`

`id` PK · `requestId` FK · `travellerId?` (null = shared group leg; set = that traveller's sub-itinerary) · `seq` · `originCode` · `destCode` · `departDate? / arriveDate?` · `transportMode` (default AIR; AIR|RAIL|OTHER) · `durationHours?` (class derivation) · `departTime?` ·

`bookingRequired` (default true) · `isPersonal` (default false; excluded from ODA) · `nights` (default 0) · `travelClassId? / entitledClassId? / chosenClassId?` (per-leg class).

4.3.4 `EstimatedExpense` (+ `AccommodationEstimate`, `ODAEstimate`)

`EstimatedExpense`: `id` PK · `requestId` FK · `travellerId?` (attributed traveller; null = shared/requestor) · `category` (AIRFARE|ACCOMMODATION|ODA|CONFERENCE|OTHER) · `expenseTypeId` (ECS master) · `currency` (default SGD) · `foreignAmount` · `sgdAmount` · `sponsorForeign` · `sponsorSgd` · `estimateBasis?` (QUOTED|CAP|LOWER|RATE) · `expectedDate?` · `autoDerived` (default false — policy-seeded per-city line, re-derived on a dates/itinerary change) · `isShared` (default false) · `shareMap?` (JSON {employeeId:pct}) · `chargingCode?` + `allocJson?` (JSON [{code,pct,io}]=100% — LINE cross-charge) · `notes?` · airfare-specific `originCode?/destCode?/proposedClassId?/fareCeiling?/handoffStatus?`. `AccommodationEstimate` (1-1, `expenseId` unique): `city` · `legSeq?` (the itinerary leg this auto per-stop line maps to) · `checkIn?/checkOut?` · `nights` · `personalNights` · `quotedNightly` · `capNightly` (from ECS policy) · `budgetedNightly` (per basis) · `capVariance` · `conferenceHotel` · `exceptionOutcome?` (PASS|EXCEPTION). `ODAEstimate` (1-1, `expenseId` unique): `country` · `city?` · `legSeq?` · `arrive?/depart?` · `eligibleDays` · `personalDays` · `dailyRate` · `ratePct` (default 100).

4.3.5 `ChargingAllocation`

`id` PK · `requestId` FK · `companyCode?` · `businessArea?` · `chargingType` (CC|WBS) · `chargingCode` (ECS master) · `isResearch` · `percent` · `amountSgd?` (AMOUNT-mode entry → percent on save) · `internalOrder?` · `isMain` (the main charging account). **Must total 100%** across the request (AC06).

4.3.6 `PolicyCheck`

`id` PK · `requestId` FK · `code` · `label` · `outcome` (PASS|WARNING|EXCEPTION|HARD_STOP) · `detail?` · `travellerId?` (per-traveller check) · `resolved` (exception approved).

4.3.7 `ApprovalStep`

`id` PK · `requestId` FK · `seq` · `roleType` (RO|DOA|RESEARCH_DOA|EXCEPTION|FUNDING_OWNER|ADDITIONAL_APPROVER|PROVOST) · `approverId?` · `onBehalfOf?` (delegation) · `status` (default Pending; Pending|Approved|Rejected|SentBack|Skipped) · `decidedAt?` · `comments?`.

4.3.8 `TravelBooking` (+ `BookingSegment`, `BookingDeviation`)

`TravelBooking`: `id` PK · `requestId` FK · `travellerId?` (group fan-out) · `tmcProviderId?` · `pnr?` · `ticketNo?` · `channel?` (TMC|SELF_BOOKED) · `fare?/taxes?/fees?/hotelRate?` · `status` (default Booked). **One TR → N bookings/PNRs** (DD-17). `BookingSegment`: `id` PK · `bookingId` FK · `type` (AIR|HOTEL) · `originCode?/destCode?` · `bookedClassId?` · `departDate?/arriveDate?` · `amount?`. `BookingDeviation`: `id` PK · `requestId` FK · `field` · `approvedValue?/bookedValue?` · `material` · `action?` (Reapproval|LogOnly|UpdateBooking).

4.3.9 TE claim entities

`TravelExpenseClaim`: `id` PK · `claimNumber` unique · `requestId?` FK · `claimantId` · `status` (Draft|Submitted) · `reportType` · `trsNotBooked` · `travelStart?/End?` · `additionalApprover1?/2?` · `lessCorpCard` · `lessPersonal` · `chargingMode` (MAIN|CLAIM|ITEM) · `chargingMainCode?` · `chargingJson?` (JSON rows + lineMap). `ClaimApprovalStep`: `id` PK · `claimId` FK · `seq` · `roleType` (VERIFIER|EXCEPTION|RO|DOA|RESEARCH_DOA) · `approverId?` · `status` · `autoGranted` (DOA auto-granted from the TR) · `decidedAt?/comments?`. `TEExpenseLine`: `id` PK · `claimId` FK · `category` · `expenseTypeId` · `treatment` (LOCKED|EDITABLE|REFERENCE|EXCLUDED) · `approvedSgd` · `bookedSgd?` (null = self-booked N/A) · `actualSgd` · `varianceSgd` · `receiptOk` · `transactionDate?` · `reason?` · `receiptType` (LOCAL_GST|FOREIGN_NO_GST|NO_RECEIPT) · `currency` · `foreignAmount?` · `sponsorSgd` · `chargingCode?/allocJson?` (ITEM allocation). `TravelExpenseLink`: `id` PK · `requestId` FK · `claimId` unique FK · `travellerId?`.

4.3.10 Supporting entities

`TravelRequestVersion`: `requestId` FK · `version` · `reason` (SUBMISSION|REAPPROVAL) · `approved` · `snapshot` (JSON) · `createdAt`. `IntegrationMessage`: `requestId` FK · `direction`

(OUTBOUND|INBOUND) · kind (TMC_HANDOFF|TMC_RESPONSE) · tmcProviderId? · payload (canonical JSON) · createdAt. *Append-only.* AuditEvent: requestId? FK · actorId? · onBehalfOf? · kind (CREATE|SUBMIT|APPROVE|REJECT|SENDBACK|AMEND|INTEGRATION|TE_LINK|STATUS|COMMENT) · summary · createdAt. Comments (§13.21) share this stream with kind=COMMENT. *Append-only.* Attachment: requestId FK · docType (Quotation|Conference Invitation|Approval Evidence|Other) · fileName · sizeKb · contentType? · content: Bytes? · uploadedById? · createdAt. Provost: id PK · schoolId · approverId · name · title · active; unique (schoolId, approverId), index schoolId. ModuleSettings (single row id SETTINGS) — §5.1. IntegrationFieldSetting (key PK · includeTmc · teTreatment) and IntegrationGuardSetting (key PK · enabled · value) — admin overrides over the §9 contract.

4.4 Status & state vocabularies

ENTITY.FIELD	VALUES (LIFECYCLE ORDER)
TravelRequest.status	Draft → Submitted → (Pending RO / Pending Additional / Pending Exception / Pending Provost / Pending DOA) → Approved → (Amendment In Progress ↻) / Cancelled / Expired / Closed; Sent Back → Draft; Withdrawn; *Rejected (legacy, non-target)*.
TravelRequest.bookingStatus	Not Sent → Sent to TMC → Received by TMC → Booking In Progress / Traveller Action Required / Partially Booked → Booked → Travel Completed; Booking Failed; Self-Booked; Cancellation Requested → Cancelled.
TravelExpenseClaim.status	Draft → Pending Verification → (Pending Exception / Pending RO) → Pending DOA → Approved; Sent Back.
ApprovalStep.status	Pending → Approved | SentBack | Skipped (Rejected legacy).
Policy outcome	PASS | WARNING | EXCEPTION | HARD_STOP.

Allowed request transitions (invalid transitions MUST NOT be offered — AC24):

```
Draft→{Submitted,Withdrawn}
Submitted→{PendingAdditional,PendingDOA,PendingException,PendingProvost}
PendingRO→{PendingDOA,PendingException,PendingProvost,SentBack}
PendingAdditional→{PendingException,PendingProvost,PendingDOA,SentBack}
PendingException→{PendingProvost,PendingDOA,SentBack}  PendingProvost→{PendingDOA,SentBack}
PendingDOA→{Approved,SentBack}  Approved→{AmendmentInProgress,Cancelled,Expired,Closed}
AmendmentInProgress→{Approved,SentBack,Cancelled}  SentBack→{Draft,Withdrawn}
Expired→{PendingRO,PendingDOA,Cancelled}  Withdrawn/Cancelled/Closed/Rejected→terminal
```

> **A Travel Request has no Reject** (DD-12): dissent = **Send Back**. `Rejected` is legacy-only.

4.5 Uniqueness & indexes

Unique: TravelRequest.requestNumber, TravelRequest.authorisationNo, TravelExpenseClaim.claimNumber, TravelExpenseLink.claimId, AccommodationEstimate.expenseId, ODAEstimate.expenseId, Provost(schoolId, approverId). Indexed: Provost.schoolId. Production SHOULD additionally index TravelRequest(status), TravelRequest(travellerId), ApprovalStep(approverId, status), AuditEvent(requestId), IntegrationMessage(requestId) for the queue/dashboard read models (§11.3).

4.6 Data volumes & retention (indicative)

Travel requests scale with staff travel (order 10^3–10^4/yr); each TR has a handful of legs/estimates/allocations/steps and 0–N bookings/segments. Audit and integration messages are the highest-volume append-only tables. Retention follows NTU records policy (travel + finance); versions and audit are retained for the authorisation's lifetime plus the statutory period. **O-R1:** confirm retention periods.

TR-PT-11 Every Travel Request MUST be answerable *as approved* from its immutable version snapshot and audit trail, independent of later edits.

Chapter 5 — Configuration model

5.1 Module Settings (runtime-togglable, AC03 — one `SETTINGS` row)

Maps to OutSystems site properties. A change persists and drives behaviour with no rebuild.

KEY	TYPE	DEFAULT	EFFECT
sameRouteResearch	bool	false	Off → research-charged requests route to the **Research DOA** line.
expenseScope	enum	ALL	`MAJOR_ONLY | ALL` — which expense types are available.
hotelEstimateBasis	enum	LOWER	`QUOTED | CAP | LOWER` — accommodation budget basis (§6.2).
airfareTreatment	enum	IN_TE	`DIRECT_SAP | IN_TE | REFERENCE` — **how** a central airfare line posts; it does **not** decide reimbursability (DD-19, §6.10).
teLinkageMandatory	enum	CONDITIONAL	`YES | NO | CONDITIONAL` — must a claim select an approved TR.
approvalAmountBasis	enum	NET_NTU	`GROSS | NET_NTU | MAJOR_COST | ALLOCATION` — amount used for DOA tiering (§6.2).
groupTravelEnabled	bool	true	Enables group (one-request-many-travellers).
groupMaxTravellers	int	10	Max travellers in a group (TMC group-fare limit); create blocked above.
groupGuestMixAllowed	bool	**false**	Groups are employees-only; guests use a separate request (DD-13).
exceptionApproverRequired	bool	true	On → a policy exception inserts an Exception Approver; off → surfaced to the DOA.
roRequirement	enum	INDIVIDUAL_ONLY	`ALWAYS | INDIVIDUAL_ONLY | NEVER` — RO decision point (DOA always required).
teAutoGrantTolerancePct	int	10	Claim DOA auto-grant tolerance, % over approved reimbursable.
teAutoGrantToleranceAbsSgd	int	500	…absolute SGD cap.
crossBaThresholdSgd	int	500	Non-primary-BA share above this must concur (§6.6).
selfBookingEnabled	bool	true	Allows Self-Booked method + TR selection at claim.
selfApprovalLimitSgd	int	0	Amount ≤ which a traveller-approver may self-approve (0 = never).
attachmentMaxMb	int	7	Max upload per file.
chargingSplitMode	enum	PERCENT	`PERCENT | AMOUNT` entry mode.
declarationText / coiText	string	(text)	Submit declaration + COI statements.
sftpConfig	string	(display)	TMC transport config (display only).

KEY	TYPE	DEFAULT	EFFECT
authorisationValidityDays	int	30	TA validity window (§13.3).
bookingDeadlineDays	int	14	Booking deadline window (§13.3).
emergencyLeadDays	int	14	DD-23 standard lead-time window; beyond it the out-of-window justification + confirmation apply.

5.2 Policy reference rates (ECS masters, read-only, effective-dated)

Hotel caps — resolve a nightly cap by city-specific cap first, else the city's **country** cap. City (SGD/night): TYO 350, OSA 300, LON 280, PAR 300, SIN 200. Country fallback: AU 260, CN 240, DE 270, EG 180, FR 290, GB 300, HK 280, IN 180, JP 320, KR 260, MY 150, SG 200, US 330. **ODA rates** (`dailyRateSgd` / `monthlyRateSgd`): AU 125/2250, CN 100/1800, DE 120/2200, EG 90/1600, FR 110/2000, GB 130/2350, HK 115/2050, IN 85/1500, JP 120/2200, KR 110/2000, MY 75/1350, US 140/2500. **ODA duration tiers:** days 1–15 → 100% of daily; 16–31 → 75%; 32+ → `monthlyRate ÷ 30` /day. **Workflow bands** (approvalAmount → DOA tier): BAND-A `0–5,000` → tier 0; BAND-B `5,000.01–20,000` → tier 1; BAND-C `>20,000` → tier 2. **Tolerances:** booking fare `+10%` or `+SGD 200` (lower); date shift `2 days`; estimate amendment `+10%` or `+SGD 500`. **Travel classes:** TC-ECO Economy (rank 1), TC-PEY Premium Economy (rank 2), TC-BIZ Business (rank 3). **Policy thresholds:** `shortLeadDays 14, longTripDays 14, highEstimateSgd 20000, severityHighSgd 2500, severityMediumSgd 500`.

5.3 Reference-master shapes (the §2 integration seam)

```
Employee{
id,name,title,departmentId,reportingOfficerId?,roles[],isTraveller,travelPolicyGroup,
          workerType?,email?,businessArea?,defaultChargingCode?,travelEligible? }
Department{ id,name,kind:SCHOOL|RESEARCH_INSTITUTE,headId,doaLine[],researchDoaLine? }
ChargingCode{
code,name,type:CC|WBS,subType?,departmentId,companyCode,businessArea,isResearch,
              active?,validTo?,fundingOwnerId?,crossCharge? }
ExpenseType{
id,teCode,name,category,glAccount,gstCode,taxRatePct,preTripApplicable,isMajorCost,
             estimateFormType:AIRFARE|ACCOMMODATION|ODA|STANDARD,varianceThreshold }
TravelClass{ id,name,rank,costIndex }   TravelClassRegisterEntry{
employeeId,entitledClassId,policyGroup,minHours,effectiveFrom,effectiveTo }
WorkflowBand{ id,minSgd,maxSgd?,doaTierIndex,label }   OdaRate{
countryCode,dailyRateSgd,monthlyRateSgd,effectiveFrom }
```

Load-bearing contracts: `Department.doaLine[doaTierIndex]` = the DOA; `Employee.reportingOfficerId` = the RO source; `ChargingCode.{departmentId,businessArea,isResearch,active,crossCharge,fundingOwnerId}` drive routing + the funding block.

5.4 Changing policy / adding a destination

A new destination or rate is **configuration only** (an effective-dated master row); no code change. A Module-Settings change is server-read on the next action. **TR-PT-12** A new destination, cap, ODA rate, workflow band or DOA line MUST be addable through configuration alone and take effect on the next evaluation with no code change or schema migration.

Chapter 6 — Runtime algorithms

Each algorithm is server-side, deterministic and re-evaluated on submit/approval. Steps are numbered; early exits noted.

6.1 Auto-estimate derivation — `syncAutoEstimates(id, resetRates)`

Policy-driven ODA + accommodation are **auto-derived one line per overnight city** from the itinerary; the traveller never hand-keys them (airfare stays manual).

1. Take the shared legs (travellerId null), ordered by `seq`.
2. For each leg *i*: `from = leg.arriveDate ?? leg.departDate`; `to = next.departDate ?? next.arriveDate ?? req.endDate`; `nights = round((to − from)/day)`. A city with `nights ≤ 0` yields no stop.
3. **Hotel:** create/update one `AccommodationEstimate` per stop at the **city cap** as the default nightly rate (`budgetedNightly` per `hotelEstimateBasis`), keyed by `(travellerId, legSeq)`. Reset an edited rate back to the cap **only when** `resetRates`.
4. **ODA:** create/update one `ODAEstimate` per city, `days = nights`, amount via the duration tiers (§6.2), keyed by `(travellerId, legSeq)`.
5. Reconcile — update matched keys, create new stops, delete stops that no longer exist. `saveTrip` calls this with `resetRates = (dates/legs signature changed)` — a dates change supersedes a prior manual rate edit. **TR-PT-13** ODA and accommodation MUST be system-derived per city from the itinerary and re-derived whenever the dates or stops change.

6.2 Cost computation

Summary (`computeSummary`): `gross = Σ sgdAmount`; `sponsorship = Σ sponsorSgd`; `major/incidental` split by `expenseType.isMajorCost`; `bookingCeiling = Σ(AIRFARE+ACCOMMODATION)`; `ntuFunded = gross − sponsorship`. **approvalAmount** = GROSS→gross, MAJOR_COST→major, NET_NTU/ALLOCATION/default→ntuFunded. `approvalAmount` is **always the full committed spend** (settlement-independent, §6.10) and drives DOA tiering. **Accommodation** (`computeAccommodation`): `cap = hotelCap(city)`; `chargeableNights = max(nights−personalNights,0)`; `budgetedNightly` = QUOTED→quoted, CAP→cap, LOWER→ `min(quoted, cap||quoted)`; `overCap = cap>0 && quoted>cap`; `capVariance = overCap?(quoted−cap)×chargeableNights:0`; `sgdAmount = budgetedNightly×chargeableNights`; `outcome = overCap?EXCEPTION:PASS`. **ODA tiered** (`computeOdaTiered({days,dailyRate,monthlyRate})`):

```
t1=min(days,15)            → t1×dailyRate×1.00
t2=max(0,min(days,31)−15)  → t2×dailyRate×0.75
t3=max(0,days−31)          → t3×(monthlyRate÷30)
sgdAmount=round2(t1Amt+t2Amt+t3Amt)
```

6.3 Travel-class entitlement — `entitledForItinerary`

1. For each flown (non-personal) leg, `entitledForDuration = register.entitledClassId if hours ≥ register.minHours else Economy` (Economy if not in the register).
2. Overall entitlement = the **highest-ranked** per-leg class (multi-city safe).
3. Persist `entitledClassId` + `classBasis` (request, per traveller, per leg).
4. A chosen class above entitlement: (a) require a justification → else hard-stop `CLASS_JUSTIFY`; (b) raise the `CLASS` exception → Exception Approver (or DOA if off); (c) for a **guest** above Economy, additionally require a selected Provost → else `CLASS_PROVOST` → Provost step (DD-25).

6.4 Policy engine — `evaluatePolicies(req, approvalAmountBasis) → PolicyCheck[]`

Each rule is gated on `ruleActive(code, onDate)` (data-driven catalogue; `mandatory` rules always run). The worst outcome gates submission (`hasHardStop` blocks; `hasException` routes).

CODE	CATEGORY	OUTCOME	TRIGGER
DATES	Dates	HARD_STOP★	Start/end missing, or end < start.
DATE_PAST	Dates	HARD_STOP	Official **start date in the past** (start-of-today cutoff; same-day OK; drafts exempt).
LEG_OVERLAP	Dates	HARD_STOP★	A leg departs a city **before the previous leg arrives** (when the prior arrive date is set).
ROUTE_GAP	Dates	WARNING	A leg's origin ≠ the previous leg's destination (possible missing connection).
ALLOC	Funding	HARD_STOP★	No allocation, or total ≠ 100%.
FUNDING	Funding	HARD_STOP★	Unknown charging code.
FUNDING_INACTIVE	Funding	HARD_STOP	A closed/inactive CC/WBS.
ESTIMATE	Estimate	HARD_STOP★	No estimate line.
HOTEL_CAP	Accommodation	EXCEPTION	Nightly rate above the city/country cap.
HOTEL_JUSTIFY	Accommodation	HARD_STOP	Above-cap hotel with **empty justification** (inside the HOTEL_CAP loop).
CLASS	Travel class	EXCEPTION	Chosen class above entitlement.
CLASS_JUSTIFY	Travel class	HARD_STOP★	Class upgrade with no justification.
CLASS_PROVOST	Travel class	HARD_STOP★	Guest above Economy with no Provost.
OFFLINE	Booking	EXCEPTION	Offline booking (needs supporting quotation + approval).
LEAD	Lead time	WARNING	Departure within `shortLeadDays` (14).
LONG	Duration	WARNING	Trip > `longTripDays` (14).
HIGH	Estimate	WARNING	Approval amount > `highEstimateSgd` (20,000).
DUPLICATE	Duplicate	WARNING	Overlapping live trip, different destination (§6.11).
HIGH_RISK	High-risk	WARNING	High-risk destination (+ mandatory traveller/approver ack).

★ = `mandatory`. **TR-PT-14** Submission MUST be blocked while any HARD_STOP is unresolved; an EXCEPTION MUST route to the configured approver; a WARNING is advisory.

6.5 Validation rules (field / cross-field / submission)

Trip step (`saveTrip`, blocks non-draft save): required — travel purpose, ≥1 leg with a destination, travel dates, booking arrangement; `end ≥ start`; **official start not in the past**; event dates within the official window; personal extension `end ≥ start` and **not overlapping** the official dates; per leg `arrive ≥ depart`; each leg within the window (widened by a personal extension); legs **chronological** (`n+1 depart ≥ n depart`); **no overlap** (`n+1 depart ≥ n arrive` when the prior arrive is set). **Date gaps between legs are the city stay and are NOT validated. Submission (`submitRequest`):** all hard-stops resolved; no **blocking duplicate** (§6.11); a destination present; a

DOA selected when candidates exist; high-risk ack when applicable; emergency gates (§6.15); no guest-attributed ODA/shared line; valid status transition; self-approval prevented (§6.7). **Claim:** no expense type absent from the approved request; a `NO_RECEIPT > 0` line raises a new exception; a claim over the approved **reimbursable** amount beyond tolerance raises a new exception.

6.6 Approval routing — `buildRoute(req, opts) → RouteStep[]`

`excluded = selfApprovalSet(req)` (every traveller + the requestor). Build in order, each step **skipping/escalating past** anyone in `excluded`:

1. **Additional Approver** — if `additionalApproverId` set and not excluded.
2. **RO** — per `roRequirement` (ALWAYS, or INDIVIDUAL_ONLY for non-group; NEVER). `resolveRoStep` walks the traveller's reporting line to the first RO not excluded.
3. **Funding Owner** — a charging account with `crossCharge && fundingOwnerId`.
4. **Cross-BA concurrence** — group allocation by business area; for each non-primary BA, if `approvalAmount × share% > crossBaThresholdSgd`, add a concurring approver (the BA's funding owner, else its DOA line), skipping excluded/already-on-route. The primary-BA DOA still owns the request.
5. **Exception Approver** — when `hasException && exceptionApproverRequired`; first `EXCEPTION_APPROVER` not excluded.
6. **Provost** — a guest class upgrade with a selected Provost (DD-25).
7. **DOA (always)** — `line = isResearch && !sameRouteResearch ? researchDoaLine : doaLine`; `tier = bandForAmount(approvalAmount).doaTierIndex`; use `selectedDoaId` when set & not excluded, else `pickDoa` walks the line from the tier escalating past excluded holders; role = `RESEARCH_DOA` or `DOA`. `statusForStep(firstPending)` sets the Pending-* status. **Emergency** builds the *same* route (no step skipped) and annotates each step "retrospective ratification (book-first, DD-23)". **TR-PT-15** Every request MUST terminate at a DOA resolved from the highest-charging department's DOA line at the amount-band tier, and MUST NOT auto-select it — the requestor chooses from the eligible candidates.

6.7 Self-approval prevention (DD-28)

No one in `selfApprovalSet` (travellers + requestor) may be an approver: single-holder steps escalate up the relevant line, multi-candidate steps pick the first eligible. Approver-only personas cannot originate a request (§8.3). **TR-PT-16** The solution MUST prevent any person approving a request they travel on or raised, by routing past them to an alternate approver, and MUST prevent an approver-only persona from originating a request.

6.8 Charging roll-up

`chargingMode` REQUEST (one split) or LINE (per-line `chargingCode` / `allocJson`). The request-level allocation driving routing is the **roll-up** of `net × %` over every line × split. `highestChargingDept` = the department bearing the largest share (tie-break to the requestor's department) → the DOA owner and cross-BA basis. "Auto-allocate by Workday department" fills each line from the attributed traveller's `defaultChargingCode`.

6.9 Authorisation issue

On the final DOA approval: set `status = Approved`; allocate `authorisationNo = TA-YYYY-NNNNNN`; set `bookingDeadline` (+ `bookingDeadlineDays`) and `authorisationExpiry` (+ `authorisationValidityDays`); snapshot an approved `TravelRequestVersion`; for a TMC arrangement, auto-hand-off to the TMC (DD-26, §6.12).

6.10 Settlement channel (DD-19) & the reimbursable rule

Single source of truth — `lineSettlement(req, category)`:

```
AIRFARE       → CENTRAL if isTmcArrangement(bookingArrangement) else EMPLOYEE   (self-booked flight = reimbursable)
ACCOMMODATION → CENTRAL if hotelSettlement==='CTA' (TMC arrangement) else EMPLOYEE
ODA / OTHER   → EMPLOYEE (always)
```

`isReimbursable = settlement===EMPLOYEE`. **Reads:** (a) TMC payload — each hotel segment `billTo = CTA|TRAVELLER`; (b) claim treatment — CENTRAL → `REFERENCE` (shown, not reimbursed), EMPLOYEE → `EDITABLE`; (c) DD-08 tolerance — compares the **reimbursable** portion on **both** sides (§6.14). `airfareTreatment` only governs *how* a central airfare line posts, never reimbursability. The hotel toggle applies to individual + group; CTA offered only for a TMC arrangement. Settlement is **separate from authorisation and cost ownership** — the approval amount is the full committed spend and the CC/WBS allocation is unchanged. **TR-PT-17** A claim line MUST be reimbursable if and only if the employee fronted it; centrally-settled lines (CTA/TMC-billed) MUST be shown for reconciliation but excluded from the reimbursable total.

6.11 Duplicate / overlap detection (DD-14) — `findDuplicateRequests(req)`

Among **live** requests (Submitted · Pending* · Approved · Amendment In Progress) other than this one, sharing an **employee** traveller (guests excluded) and whose **official dates overlap**: `sameDestination = sameCountry && (same city when both set)`. **True duplicate** (same dest) ⇒ hard block at submit; **date clash** (different dest) ⇒ warning. **Surfaced upfront** on every editable step (Trip·Estimates·Charging·Policy) once dates are known; the submit-stage block is the backstop. **TR-PT-18** The solution MUST block a second live authorisation for the same traveller + overlapping dates + same destination, surfaced from the first editable step, and MUST warn on an overlapping different-destination trip.

6.12 TMC hand-off, group fan-out & multi-PNR

Hand-off (`handoffToTmc`): build the canonical outbound payload (§9.1) honouring the Integration Contract; send only when the outbound guards hold (Approved · Not Sent · TMC-served method · TA set); emergency sends on submit (DD-26); **idempotent** (never re-send an in-flight/booked request). **Group fan-out:** one instruction per traveller, each with that traveller's effective itinerary/class/nights/share. **Multi-PNR:** a `≥3`-air-segment multi-city trip is split into PNR-A (first half + hotel) and PNR-B on inbound; each PNR is its own `TravelBooking`.

6.13 Inbound — TA-keyed retrieval & deviation (`receiveBooking`, `refreshBookingsFromTmc`, `bookingsForAuthorisation`)

The claimant creates the claim **from the TR**; the TMC exposes retrieval **by the TA number**, returning the **full PNR set** (not a PNR-by-PNR lookup). `bookingsForAuthorisation(req)` → `{authorisationNo, pnrCount, bookings[{pnr,ticketNo,providerId,travellerId,status,fareSgd,taxesSgd,feesSgd,hotelRateSgd,totalSgd,segments[{type,origin,dest,departDate,arriveDate,bookedClassId,amount}]}], totalFareSgd}`. **Push-primary** (receiveBooking stores bookings against the TR so the claim pre-fills with zero PNR entry) + a **pull-by-TA refresh** (idempotent). A self-booked trip returns an empty set. **Deviation:** booked aggregate airfare vs approved + tolerance (`+10%` / `+SGD 200`, lower) → material deviation → re-approval. `legPnrMap(req)` maps each AIR leg to its PNR (origin+dest + date) for per-leg provenance on the TR (§11). **TR-PT-19** Booking retrieval MUST be keyed by the Travel Authorisation (returning all PNRs under it), not the PNR; captured actuals MUST include booked fares and booked dates.

6.14 Claim prepopulation & DD-08 auto-grant (`prepopulateClaim`, `buildClaimRoute`, `doaAutoGrant`)

Prepopulate: header mostly LOCKED; each expense line's treatment = `REFERENCE` when `!isReimbursable` (TMC airfare, CTA hotel) else `EDITABLE`; admin contract override wins; EXCLUDED dropped; group filtered to the claimant's lines; shared accommodation costed on the claimant's own nights; prior-claim amounts netted per category (AC17). **Route:** Verifier → (Exception if a new exception) → (RO per `roRequirement`) → DOA (always; from the claim's effective charging dept + band). **Auto-grant** (`doaAutoGrant`) — all must hold: TR Approved/Closed

with a TA; `claimTotal − approvedAmount ≤ tolerance` (% and SGD); no new expense type; no new exception; **unchanged charging DOA** (a changed account routes to the new account's DOA, never auto-granted). Scope: `claimTotal = Σ non-REFERENCE lines − corpCard − personal` ; `approvedAmount = Σ reimbursable (EMPLOYEE) approved expenses net of sponsorship` — **like-for-like**, central airfare + CTA hotel excluded from both sides.

6.15 Emergency lane (DD-23)

Book-first/raise-later, bounded & audited. Submit gates: a named **authorising officer** (not a traveller) + the authorising **e-mail attached** (mandatory evidence); **agent-assisted/offline TMC only**; destination mandatory; policy hard-stops still apply; soft-gated by `emergencyLeadDays` — outside the window a detailed justification (≥ min length) + an explicit confirmation are required and the request is flagged *Outside standard window*. The **full route still runs** (same approvers, ratified retrospectively, §6.6); on submit the instruction is auto-handed to the TMC with an `EMERGENCY_BOOK_FIRST` indicator. **TR-PT-20** The emergency lane MUST change only the *timing* of approval, never the set of approvers, and MUST require a named authoriser + attached authorising e-mail; it MUST be available only for agent-assisted/offline TMC bookings.

Chapter 7 — Lifecycle & change governance

7.1 Request lifecycle

Draft → Submit (route built) → approvals (each `approveStep` advances or finalises) → **Approved + TA** → TMC hand-off → Booked → Travel Completed → TE claim. Dissent at any step = **Send Back** (→ Sent Back → Draft) with a reason; there is no Reject (DD-12).

7.2 Amendment

A material change to an Approved request moves it to **Amendment In Progress**, re-derives the route and re-approves, issuing a revised TA. The amendment threshold is `estimateAmend` tolerance (`+10%` / `+SGD 500`).

7.3 Failed-booking recovery (§6.4 / DD-15)

The OBT reports booking **status**, not an approval. Not issued within the window → **Booking Failed**; recovery (gated by the window): **re-send** (retry), **amend & re-route** (reopen → Amendment In Progress, single-TA path), **cancel & re-raise** (original terminal, frees the overlap under DD-14). A fare over tolerance routes to re-approval.

7.4 Cancellation (DD-22)

Before hand-off → local withdraw/cancel. After transmit/book → via the TMC (`cancelRequest` → Cancellation Requested → `receiveCancellation` → Cancelled). This + the duplicate guard prevents a double-booking on a later re-raise.

7.5 Versioning & audit

Every submission/approval snapshots an immutable `TravelRequestVersion` (`currentVersion++`). Every field/workflow/integration action and every comment is an append-only `AuditEvent` . **TR-PT-21** Amendment of an approved request MUST re-run the full approval route and re-issue the TA; the prior authorisation version MUST be retained.

Chapter 8 — Security & access control

8.1 Three mechanisms, kept separate

(1) **Creation remit** — may this persona originate a request, and for whom. (2) **Edit remit** — may this persona edit a draft. (3) **Approver entitlement** — may this persona act on *this* step. They are independent; holding one never implies another.

8.2 Identity modes

Prototype: a persona cookie (`ecs_persona` + `ecs_role`) drives `currentPersonaId()` / `currentRoleId()` ; the switcher groups personas by stage (REQUEST / CLAIM / ADMIN). **Production MUST** replace this with ECS SSO/identity; the persona id becomes the authenticated employee id and roles come from ECS. **TR-PT-22** Production MUST derive identity and roles from ECS authentication, not a client-set value.

8.3 Creation remit — `creatableTravellerIds(persona)` / `canCreateTravelRequest`

A persona may raise for: **themselves** (if a traveller); **principals who delegated** to them (`EcsDelegation.canCreateTravelRequestFor`); for a **Travel Requestor/PA** — travellers in their department; for the **Travel Administrator** — any traveller (agent-assisted). **No catch-all fallback:** an approver-only persona (DOA/Research DOA/Exception), System Administrator or Finance viewer gets an **empty set** and cannot originate a request (DD-28 / separation of duties).

8.4 Edit remit — `canEditRequest(persona, req)`

True for: Travel/System Administrator; the creator (`requestorId`); the traveller themselves; a delegate for the traveller; a Travel Requestor in the traveller's department. **Explicit remit only** — read-only roles and approvers are excluded.

8.5 Approver entitlement & separation of duties — `canActOnStep` , `isApproverPersona`

An approval runs only when `approverId && approverId === persona` — the acting persona **is** the step's assigned approver. **A Travel/System Administrator does NOT get an approval bypass** — they may approve only when they independently hold the role and are the assigned approver. A step with **no resolved approver is not open** to any persona (closes the null-approver hole). The RO/DOA dashboard is gated by `isApproverPersona` (admins are not approvers). **Self-approval** is prevented in routing (§6.7), not just the UI.

8.6 Lifecycle guards — `actionAllowed(action, status)`

Every mutating action checks a valid **source state** before running: `submit` ∈ {Draft, Sent Back}; `approve` / `sendBack` ∈ {Pending RO/Additional/Exception/DOA, Amendment In Progress}; `withdraw` ∈ {Draft, Sent Back, Pending*}; `reopen` ∈ {Sent Back}; `cancel` ∈ {Approved, Amendment In Progress, Expired}. There is **no reject** action.

8.7 Data scope & audit

The dashboard is **traveller-scoped** (a persona sees their own + delegated + department requests; administrators see all; the approver queue shows steps assigned to the acting persona). Finance is read-only oversight. Every action writes an `AuditEvent` ; audit/integration are append-only. **TR-PT-23** Every mutating action MUST re-check the source state and the acting remit server-side; the client MUST NOT be trusted to have gated it.

Chapter 9 — Integration & external interfaces

9.1 Principles

One governed boundary per external system; the module **reads** masters and **sends** governed, field-filtered instructions; inbound responses are status, never authority. All outbound/inbound payloads are canonical JSON persisted as `IntegrationMessage` rows. The Integration Contract is **declarative and admin-overridable**.

9.2 System inventory

SYSTEM	DIRECTION	FLOW
ECS / HR identity, DOA, Workday	Read	Employees, reporting lines, DOA lines + bands, home-dept cost object.
SAP Finance	Read + Write*	CC/WBS, GL, GST, IO (read); approved allocation → GL (*via claim/finance IF); DIRECT_SAP airfare posting.
Policy masters / FX / locations	Read	Caps, ODA, class register; effective-dated FX; IATA airport master.
TMC / OBT	Outbound + Inbound	Governed booking instruction (field-filtered; one per traveller for a group) ⇄ booking status + **PNR set keyed by TA**.
Immigration / Visa	Outbound	Daily visa-letter batch for approved visa-flagged trips.
Notifications	Outbound	Approval/action/booking/deadline alerts.
ECS TE Claim	Outbound (seed)	Approved TR pre-populates the claim; reconciles to the TA; DOA auto-granted within tolerance.

9.3 Outbound payload (Send to TMC) — `CONTRACT_FIELDS`

Core (always): `authorisationNumber, travellerRef, bookingMethod`. **Toggleable (default on):** group `travellers`, `bookingDeadline`, `costCeiling`, `approvedExceptions`, `tripPurpose`, `eventDates`, `approvalValidity`, `airSegments` (origin/dest/depart+arrive), `airApprovedClass`, `hotelSegments` **(city/nights/cap +** `billTo` **CTA|TRAVELLER)**, `charging` (CC/WBS/BA), `approvalMetadata`, `travellerCategory` +contact, `personalIndicator`, `guestProfile` (PDPA secure-link, guest-only). Emergency adds the `EMERGENCY_BOOK_FIRST` indicator. **Guards (** `CONTRACT_GUARDS` **):** OUTBOUND — status=Approved ∧ bookingStatus=Not Sent ∧ TMC-served method ∧ TA set. INBOUND — in-flight ∧ TA set. TE_SYNC — status ∈ {Approved, Closed} ∧ TA set.

9.4 Inbound record shape (TA-keyed)

See §6.13 — `bookingsForAuthorisation` return shape. Group fan-out returns one booking per traveller; multi-city splits across PNRs; self-booked returns an empty set. Deviation is checked on the aggregate airfare.

9.5 Interface requirements

TR-PT-24 The TMC interface MUST accept a field-filtered governed instruction and expose booking retrieval by TA (returning all PNRs), and MUST support a group fan-out (one instruction per traveller). **TR-PT-25** Guest passport/profile MUST be collected by the TMC via a secure link; ECS MUST store only guest name/email/organisation (PDPA, DD-21). **TR-PT-26** All external hand-offs MUST be idempotent and fully audited as `IntegrationMessage` rows.

Chapter 10 — Service & action catalogue

10.1 Conventions

No REST layer in the prototype — **mutations are server actions** (`"use server"`), **reads** are server-component loaders (§11.3). Each action: resolves persona → checks source state + remit → transacts → audits → `revalidatePath` / `redirect(?error=)`. Production MAY expose these as Service Actions / REST.

10.2 Integration seam (ECS services — the boundary to wire)

`EcsIdentity` (employee, department, reportingOfficer, hasRole), `EcsRoles` (employeesWithRole), `EcsDelegation` (canCreateTravelRequestFor), `EcsReference` (expenseType, travelPurpose, country/city/airport, travelClass, currencies/fx, flightHours, highRisk, rejectionReasons), `EcsFx` (sgdPerUnit/toSgd, effective-dated), `EcsPolicy` (hotelCap, odaRate/odaMonthlyRate, tolerances, ODA-tier constants), `EcsTravelClassRegister` (entry, entitledForDuration/Itinerary, rank), `EcsCharging` (code), `EcsWorkflow` (bandForAmount). Contracts in §5.3.

10.3 Server actions (pre-trip) — contract

ACTION	GUARDS	EFFECT
createDraft / createGuestDraft / createGroupDraft	creation remit (§8.3)	New draft (individual / guest / group employees-only).
addGroupTraveller / removeGroupTraveller	edit remit	Manage group (≤ `groupMaxTravellers`).
saveTrip	edit remit; §6.5 validation	Persist trip/dates/itinerary; re-derive auto estimates (§6.1).
addLeg / removeLeg / moveLeg	edit remit	Itinerary edits.
addAirfare / addAccommodation / addOda / addOther	edit remit; entitlement-scoped	Add estimate lines (no ODA for guests).
setHotelSettlement	edit remit; TMC-arrangement for CTA	DD-19 CTA/Individual (§6.10).
updateAccommodationRate	edit remit	Inline hotel rate (above-cap justification).
deleteExpense	edit remit	Remove a manual line.
saveCharging	edit remit	Allocation split (=100%).
runPolicy	edit remit	Re-run engine; persist checks.
submitRequest	§6.5 submission gates + duplicate + self-approval	Build route; set Pending-*.
approveStep	`canActOnStep` (assigned approver)	Advance or finalise (issue TA + auto hand-off).
sendBack	assigned approver	→ Sent Back (reason).
withdrawRequest / reopenDraft	edit remit + source state	Draft/sent-back lifecycle.
cancelRequest / receiveCancellation	source state	§34 cancellation (TMC-mediated after hand-off).
handoffToTmc	outbound guards; idempotent	Transmit the instruction.
receiveBooking	inbound guards; idempotent	Ingest booking(s) — fan-out + multi-PNR + segments + deviation.
refreshBookingsFromTmc	TA set; idempotent	TA-keyed pull.
setBookingTmcStatus / markSelfBooked	source state	Lifecycle / self-booked.
postAirfareToSap	DIRECT_SAP + booked	SAP airfare posting.

ACTION	GUARDS	EFFECT
resendToTmc / amendAfterFailedBooking	failed state	§6.4 recovery.
runVisaLetterBatch	—	Daily visa notification.
addComment / addAttachment	edit/comment remit; ≤ `attachmentMaxMb`	Comment feed / evidence.
copyToNewRequest	creation remit	Clone to a new draft.

TE claim: `createClaimFromRequest(requestId)` (TE_SYNC guards; prepopulate), `updateClaimActuals`, `submitClaim` (build claim route + auto-grant), `approveClaimStep`, `sendBackClaim`, `updateClaimHeader`, `addClaimExpense`, `removeClaimExpense`, `saveClaimCharging`.

10.4 Key call chains

Submit: `submitRequest` → `evaluatePolicies` + `findDuplicateRequests` → `computeSummary` → `buildRoute` → persist steps + status. Final approval: `approveStep` → issue TA → `handoffToTmc` → `assembleOutbound` / `buildOutbound`. Inbound: `receiveBooking` → `simulateInbound` (adapter) → split → persist bookings/segments → deviation. Claim: `createClaimFromRequest` → `prepopulateClaim` (settlement treatment) → `buildClaimRoute` → `doaAutoGrant`.

10.5 Separation-of-duties / integrity checklist (where enforced)

CHECK	WHERE
No create by approver-only persona	`creatableTravellerIds` (empty set)
No self-approval	`selfApprovalSet` in `buildRoute`
No admin approval bypass	`canActOnStep` (`approverId===persona`)
No null-approver step open	`canActOnStep` (`!!approverId`)
Valid source state	`actionAllowed`
100% allocation	`ALLOC` hard-stop
Hard-stop blocks submit	`hasHardStop` in `submitRequest`
Like-for-like tolerance	`doaAutoGrant` reimbursable scope
Idempotent hand-off/inbound	`handoffToTmc` / `receiveBooking`

Chapter 11 — Screen catalogue

11.1 Global shell

Persona switcher (role + person), role-gated left nav, request/claim/admin stages. Wizard stepper: **Traveller** → **Trip** → **Estimates** → **Charging** → **Policy Review** → **Submit** (soft-save; validation deferred to Submit).

11.2 Route catalogue

ROUTE	SCREEN	FIELDS / ACTIONS / VALIDATION
/dashboard	Dashboard	Traveller-scoped; grouped by drafts/pending/approved-unbooked/completed-unclaimed; search/type/status; S/E flag.
/requests/new	Create	Individual/group/guest; creation-remit gated.
/requests/[id]	Traveller	Traveller + delegate/PA context.
/requests/[id]/trip	Trip	Purpose; booking arrangement + method; resolved TMC; dates (official/event/personal); itinerary builder; visa flag; **upfront duplicate banner**; `saveTrip` (§6.5).
/requests/[id]/estimates	Estimates	Auto per-city ODA+hotel; **hotel-settlement toggle** (TMC-gated); add airfare/accom/ODA/other; inline hotel rate; duplicate banner.
/requests/[id]/charging	Charging	Request/line split (=100%); auto-by-department; PERCENT/AMOUNT.
/requests/[id]/policy	Policy Review	Run engine; outcomes; re-run; duplicate banner.
/requests/[id]/review	Review & Submit	One-page summary (Trip incl. **PNR provenance** once booked; Charging; Estimates; route preview + DOA selector); declaration + COI; high-risk ack; emergency panel; hard-stop/blocking-duplicate disable Submit; `submitRequest`.
/requests/[id]/approve	Approver/DOA	One-page Approval Summary; Approve / Send Back.
/requests/[id]/booking	Booking & TMC	Send/receive/set-status/self-booked/re-send/amend/cancel; per-PNR cards; ↻ **Refresh from TMC**; SAP posting.
/requests/[id]/pack · /details	Pack / Full details	Printable pack; full detail cards.
/claims/[id]	TE claim	Pre-filled from TR; actuals; would-auto-grant; submit.
/approvals	RO/DOA dashboard	Travel-Requests/Expense-Claims tabs; per-approver counters; exception shading.
/workbench · /finance · /reports	Admin / Finance / Reporting	Overrides/ops; read-only oversight; aggregates.
/visa-letters · /notifications · /integration · /config-viewer · /settings · /setup/provosts	Admin	Visa batch; notifications; contract editor + payload preview; reference viewer; Module Settings (SysAdmin); Provost list.

11.3 Data loaders / read models

`loadRequest(id)` eager-loads the full aggregate (travellers, legs, expenses+accommodation/oda, allocations, policyChecks, approvalSteps, bookings+segments, deviations, teLinks, claims, messages, auditEvents, attachments). `listRequests()` is the dashboard read model. Production SHOULD back the dashboard/queue with the indexes in §4.5.

Chapter 12 — Reference data & seed

12.1 Reference-data inventory (`src/data/*` , read through §2)

`employees` , `organisation` (departments/entity), `charging` (CC/WBS), `expenseTypes` , `travelPurposes` , `locations` (countries/cities/airports + SIN flight-hours), `policyRates` (hotel caps + ODA + tiers), `travelClass` + `travelClassRegister` , `workflow` (bands), `fxRates` , `tolerances` , `delegations` , `rejectionReasons` , `highRiskDestinations` , `tmcProviders` , `personaRoles` , `visaLetter` . These are the masters a production build wires to live ECS.

12.2 Seed scenarios (demo, 28) — lazy-seeded when empty

S01 standard trip (approved, booked, claimed) · S02 research trip (Research DOA) · S03 above-cap hotel (exception, Pending Exception) · S11 group conference (PA raises three) · S12 group amendment (loses a traveller → reapproval) · S14 multi-leg (Tokyo+Osaka, personal day) · S15 class upgrade exception · S16 multi-country (London+Paris) · S18 emergency book-first (Seoul) · S19 a draft parked · S20–S23 pending-stage requests across trip types · S22 multi-city pending · S23 one-way pending · S24 long-term secondment (45 nights → ODA tiers) · S25 guest Seoul pending · S26 guest London booked · S27 emergency-eligible draft at submit · **S28 multi-city booked across two PNRs** (per-leg PNR provenance). The seed asserts no hard-stop at submit (bad seed data fails the build) and exempts historical-dated fixtures from DATE_PAST.

Chapter 13 — Verification & acceptance

13.1 Principles

Acceptance is behaviour-first: each rule/algorithm is demonstrable in the running prototype against seeded policy data. The prototype is the executable specification; a respondent's build is accepted when it reproduces the behaviours below.

13.2 Acceptance test catalogue (illustrative)

AREA	ACCEPTANCE
Entitlement	Multi-city class = highest per-leg entitlement; upgrade without justification is a hard-stop.
Cost	ODA follows the duration tiers (e.g. 45 nights = 15×100% + 16×75% + 14×monthly÷30); above-cap hotel is capped + exception + mandatory justification.
Policy	Each rule in §6.4 fires on its trigger; hard-stops disable Submit.
Validation	Past start, leg overlap, route gap, non-100% allocation, inactive account each behave per §6.5.
Routing	Route = Additional?→RO?→Funding/Cross-BA?→Exception?→Provost?→DOA; self-approver escalated; approver-only persona cannot create.
Settlement	Airfare + CTA hotel are REFERENCE; individual hotel + ODA reimbursable; self-booked airfare reimbursable; approval amount unchanged.
Duplicate	True duplicate blocks submit (surfaced upfront); date clash warns.
TMC	TA-keyed retrieval returns all PNRs; multi-city splits into 2 PNRs; hand-off/inbound idempotent; booked arrive dates captured.
Claim	DOA auto-granted within tolerance; changed account routes to the new DOA; reimbursable scope like-for-like.

13.3 Automated test inventory

The prototype has **no automated test suite** (verification is behavioural, via seeded scenarios). **TR-PT-27** A production build SHOULD provide an automated acceptance suite covering the §13.2 catalogue (entitlement, cost tiers, policy outcomes, validation, routing/SoD, settlement, duplicates, TMC, claim auto-grant).

Chapter 14 — RFP / production considerations

14.1 Using this document

Chapters 4–6 and 9–11 are the build contract; Chapter 14.2 is where the prototype simplifies; Appendix D records binding design decisions; Appendix E indexes the derived requirements.

14.2 Prototype-to-production gap register

#	AREA	PROTOTYPE	PRODUCTION REQUIREMENT
G-01	Identity	Persona cookie switcher	ECS SSO/identity; roles from ECS (TR-PT-22).
G-02	TMC	Inbound simulated by an adapter	Real TMC/OBT interface (outbound + TA-keyed inbound), idempotent, audited (TR-PT-24/26).
G-03	Money	SGD `Float`	Exact decimal (TR-PT-04).
G-04	Storage	SQLite; attachments in-row	Production RDBMS; document store (bytes out of the row).
G-05	API	Server actions only	Expose Service Actions / REST as required.
G-06	Structured values	JSON-in-string	JSON/JSONB or normalised child tables.
G-07	Versioning	Snapshot-on-submit/approve	Confirm full field-level version history + as-at enquiry (TR-PT-09/11).
G-08	SAP / visa / notifications	Modelled as actions/audit	Wire to SAP (GL + airfare posting), immigration batch, notification channel.
G-09	Tests	Behavioural only	Automated acceptance suite (TR-PT-27).
G-10	FX	Monthly table	Live effective-dated FX, re-locked at submission (TR-PT-06).

14.3 Indicative non-functional requirements (University to complete `[ ]`)

- **Performance:** policy+route evaluation p95 `[ ]` ms; dashboard/queue load p95 `[ ]`; concurrent users `[ ]`.
- **Availability / RPO / RTO:** `[ ]`.
- **Volumes:** requests/yr `[ ]`; peak group size (TMC limit `10`); audit/integration retention `[ ]` (O-R1).
- **Security:** SSO, role-based access, separation of duties (Ch 8), full audit; PDPA for guest data (TR-PT-25).
- **Accessibility / i18n:** `[ ]` (NTU standards).
- **Integration SLAs:** TMC hand-off / inbound latency `[ ]`; failure/retry semantics.

14.4 Suggested deliverables & acceptance

Source + migrations + seed; the §13.2 acceptance suite; the integration adapters (TMC, SAP, HR/DOA, visa, notifications); the configuration screens; and a demonstration reproducing the seeded scenarios (Ch 12).

14.5 Information to request from respondents

TMC/OBT interface capability (TA-keyed retrieval, group fan-out, secure guest-profile link); SAP GL/airfare posting interface; HR/DOA/Workday read interfaces; identity/SSO integration; data-migration approach; the proposed test strategy against Ch 13.

Appendix A — Glossary

TR Travel Request · **TA** Travel Authorisation · **DOA** Delegation of Authority · **RO** Reporting Officer · **ODA** Overseas Daily Allowance · **TMC** Travel Management Company · **OBT** Online Booking Tool · **PNR** Passenger Name Record · **CTA** Central Travel Account · **CC/WBS** Cost Centre / Work Breakdown Structure · **BA** Business Area · **TE** Travel Expense (claim) · **SoD** Separation of Duties · **PDPA** Personal Data Protection Act.

Appendix B — Enumerations & constants

Request status, booking status, claim status, policy outcome, approver role, booking method/arrangement, expense category, TE treatment, ECS role — **full value lists in §4.4 and §4.3**. Band bounds `(lower, upper]` with first lower bound 0 inclusive. Tolerances, caps, ODA rates + tiers, workflow bands, policy thresholds — §5.2.

Appendix C — Source map & traceability

Routing `route.ts` · policy `policy.ts` + `policyRules.ts` / `policyThresholds.ts` · cost `pricing.ts` · settlement `settlement.ts` · duplicates `duplicates.ts` · TMC `tmcPayload.ts` / `tmcRetrieval.ts` / `integrations/tmc/*` · guards `guards.ts` · session `shared/session.ts` · ECS seam `shared/ecs/services.ts` · settings `config/decisions.ts` · rates `data/policyRates.ts` · actions `modules/pretrip/actions.ts` + `modules/te/*` · seed `modules/pretrip/demo.ts`. Design decisions trace to the companion Build Spec (DD-01…DD-29).

Appendix D — Decision register (design decisions)

DD-01 config-driven routing · DD-02 DOA always/traveller-selected · DD-03 RO configurable · DD-04/05 class entitlement; group per-traveller · DD-06 accommodation never shared · DD-07 per-line charging · DD-08 claim auto-grant · DD-09 one itinerary/trip-type preset · DD-10 arrangement vs method · DD-11 airport master = TMC source · DD-12 no Reject (send back) · DD-13 groups employees-only · **DD-14** one live authorisation (duplicate upfront) · DD-15 failed-booking recovery · DD-16 estimate defaults · DD-17 1 TR→N PNRs; **TA-keyed inbound + PNR provenance** · DD-18 online vs offline hand-off · **DD-19** settlement channel; reimbursable = employee-settled · DD-20 upgrade base/differential split (O-5) · DD-21 guest PDPA via TMC · DD-22 cancellation local-then-TMC · DD-23 emergency lane · DD-24 dashboard approvals + one-page summary · DD-25 guest upgrade → Provost · DD-26 automatic TMC hand-off · DD-27 destination mandatory · DD-28 separation of duties. **Open:** O-1 auto-grant tolerance figures · O-5 upgrade split percentages · O-R1 retention periods.

Appendix E — Index of derived requirements

TR-PT-01 full lifecycle server-enforced · 02 single integration boundary · 03 no admin approval bypass · 04 exact money · 05 server-side evaluation · 06 effective-dated FX re-locked · 07 audit + version snapshots · 08 idempotent hand-off/inbound · 09 immutable versions · 10 append-only audit · 11 as-approved from snapshot · 12 config-only policy change · 13 per-city auto-derived & re-derived estimates · 14 hard-stop/exception/warning semantics · 15 DOA always, not auto-selected · 16 no self-approval; no approver-origination · 17 reimbursable iff employee-fronted · 18 duplicate block + overlap warn, upfront · 19 TA-keyed retrieval + captured actuals · 20 emergency timing-only + named authoriser · 21 amendment re-runs route + re-issues TA · 22 production identity from ECS · 23 server-side state + remit re-check · 24 TMC interface (field-filtered, TA retrieval, fan-out) · 25 guest PDPA via TMC · 26 idempotent + audited hand-offs · 27 automated acceptance suite.

This specification describes the intended build. Final design may change at a formal discovery / detailed-design phase with the implementation vendor; material changes are tracked by version.

Implementation-level technical reference derived from the reference prototype. Statements marked “Derived requirement (TR-PT-nn)” are subject to University confirmation. Material changes are tracked by version.